

Software Architecture
(T630019402)

Lecture #8, April 14, 2026

Jukka Ruohonen

Survey (1/4)

- ▶ There is now a feedback survey on *itslearning*
- ▶ Although it is not anonymous, I would appreciate if you would answer as it helps me to plan for the next year
- ▶ As we are in a university, after all, if enough answer, it also gives some interesting data

Survey (2/4)

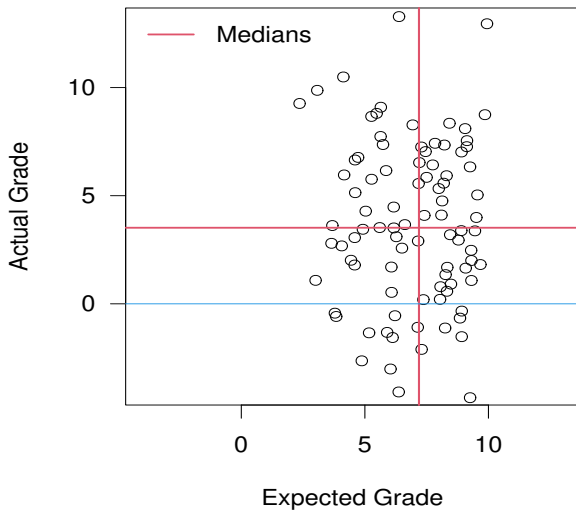


Figure 1: The First Tentative Hypothesis (yet not fully accurate)

Survey (3/4)

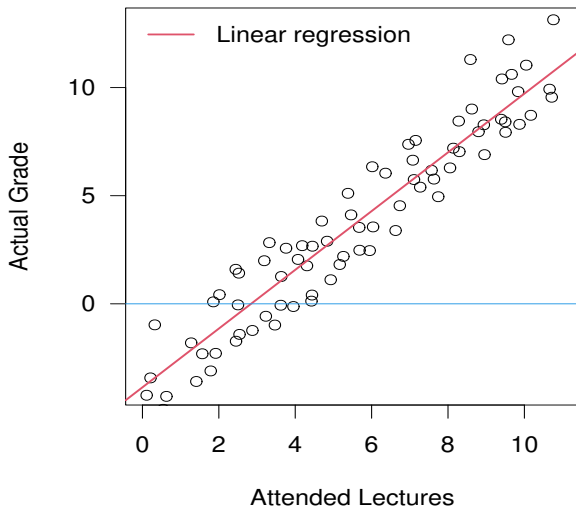


Figure 2: The Second Hypothesis (yet not fully accurate)

Survey (4/4)

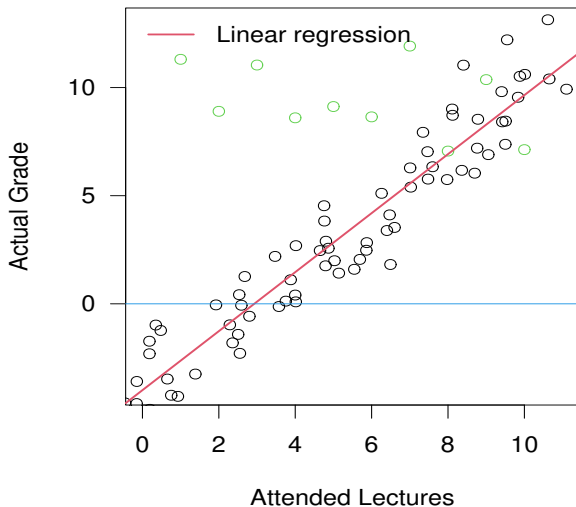


Figure 3: The Second Hypothesis Adjusted (yet still not fully accurate)

Case #7

- ▶ Design an alert system with Prometheus for providing *instant notifications* to a system administrator in case *two or more* Kubernetes *nodes* or pods become dead while in operation
 - Illustrate the design with **two architectural views**: one addressing the design at the side of Kubernetes and the other at the Prometheus' side
 - The design does not have to be 100% accurate technically; rather, consider that it would be a **blueprint** that you would provide as a software architect to an implementation team
 - Rationalize also other QAs, such as complexity

A Few Short Mock-up Questions Again (1/7)

What Does the Amdahl's Law Say About Scalability?

A Few Short Mock-up Questions Again (2/7)

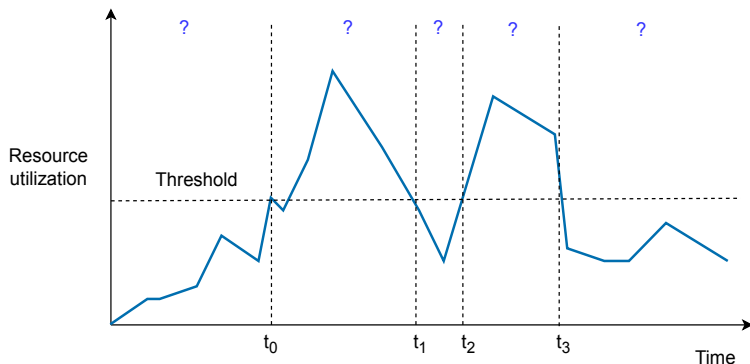


Figure 4: (a) What the Question Marks Might Denote in Terms of a Circuit Breaker Measuring Resource Utilization, and (b) How Would You Calculate the Time Spent During the Circuit Breaker's States?

A Few Short Mock-up Questions Again (3/7)

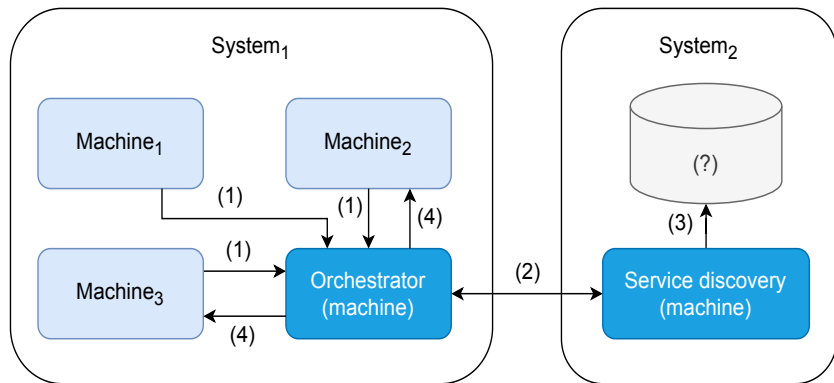


Figure 5: (a) What (1), (2), (3), (4), and (?) Might Denote, and
(b) What Might Be the Case With Machine₁?

A Few Short Mock-up Questions Again (4/7)

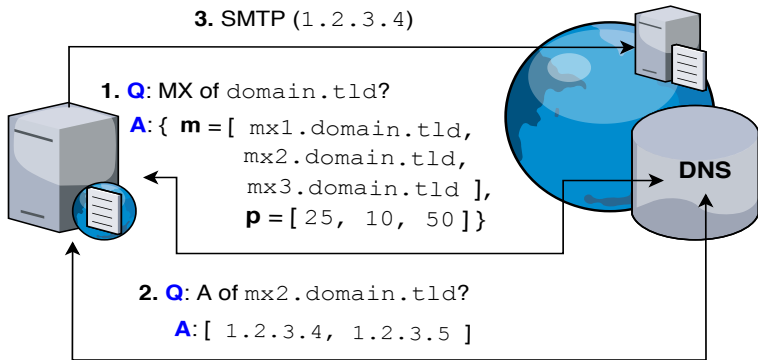


Figure 6: (a) Why the Client Picked `mx2.domain.tld`, (b) Which Architectural Pattern Is Present in the Choice, (c) Why the Client Picked `1.2.3.4`, and (d) Which Architectural Pattern Is Present in This Choice?

A Few Short Mock-up Questions Again (5/7)

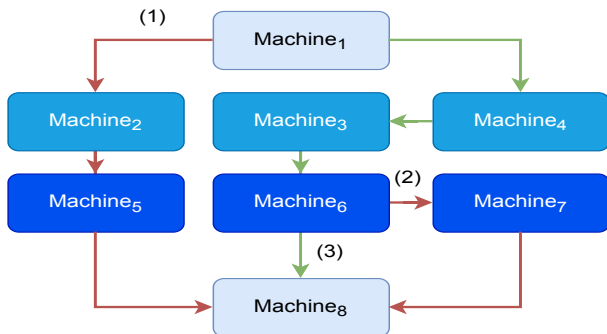


Figure 7: (a) What Architectural Tactic Is Used in the Diagram, (b) What (1), (2), and (3) Might Denote, and (c) What Might Be Logically Deduced About Latency Between Them?

A Few Short Mock-up Questions Again (6/7)

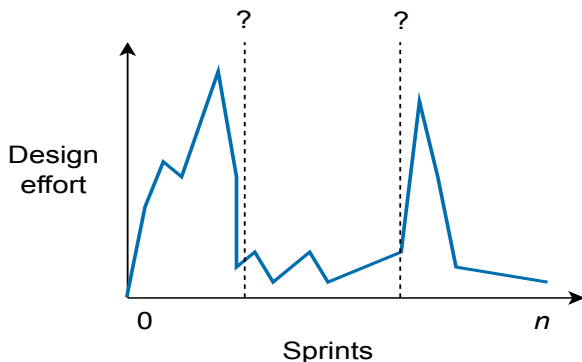


Figure 8: (a) What Might Have Happened in This Agile Project, and
(b) What the Question Marks Might Denote to Make It Explicit?

A Few Short Mock-up Questions Again (7/7)

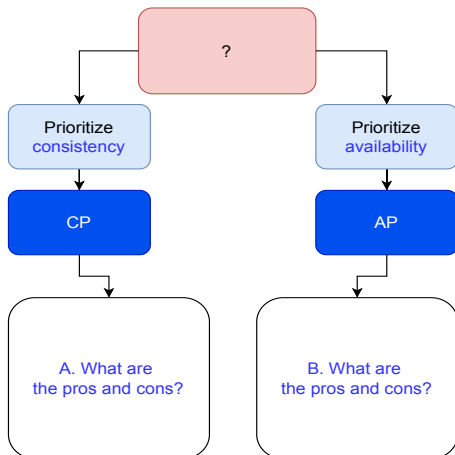


Figure 9: (a) Please Explain What the Question Mark Represents, and
(b) Answer to A and B With *Six Sentences at Most*

Today's Lecture

- ▶ Today's lecture is about **safety** and **security** as distinct QAs
- ▶ Though – given that many (or even most) of the safety tactics and patterns are similar (or even identical) with those for availability and fault-tolerance, safety is considered only briefly
 - Rather, it is important to understand safety's fundamental relation to the impact dimension
- ▶ Then, cyber security is covered also in the next lecture
 - It is assumed that you already know (as you should) the fundamentals, starting from the CIA triad

Question

How would you define safety and why is it different from security?

Safety

- ▶ Safety *“is concerned with a system’s ability to avoid straying into states that cause or lead to damage, injury, or loss of life to actors in its environment”* (Bass et al. 2021)
- ▶ The definition does not say anything about why a system is entering into a dangerous state; hence, a conventional way to separate safety from security is that the latter always involves an **actor** who **intentionally** seeks to cause damage or harm
- ▶ Further note that **insecurity may cause safety issues**, but the reverse is more difficult to deduce about (or even untrue according to some people)

Safety Scenarios

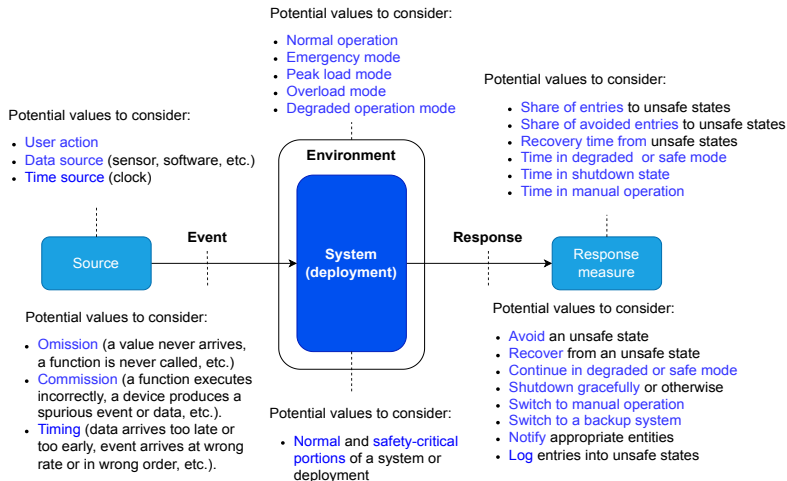


Figure 10: Adopted and modified from Bass et al. (2021)

Safety Tactics (1/2)

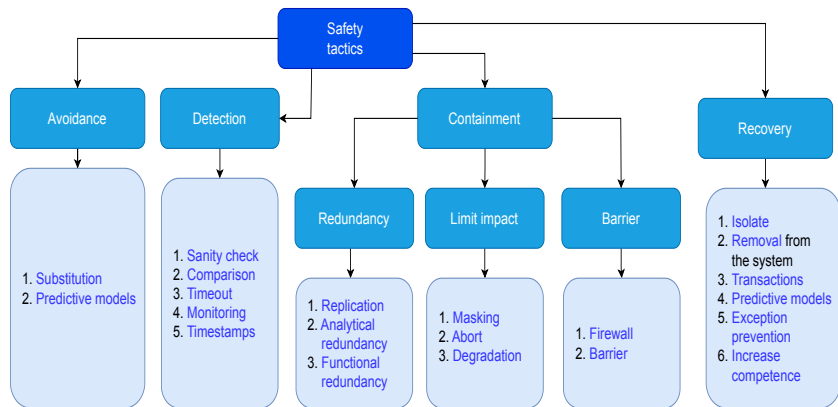


Figure 11: Adopted and modified from Bass et al. (2021)

Safety Tactics (2/2)

- ▶ Thus, all of the tactics have already been covered, although perhaps with a slightly different terminology
 - For instance, the **barrier tactic** is about limiting the propagation of unsafe states, and the firewall is a security-specific variant noted already in the second lecture
- ▶ Of the impact-limiting tactics, **aborting** is the *de facto* tactic for safety, but recall also the earlier points about **fail-safe**, **fail-fast**, or **crash** and **panic**

Safety Patterns (1/2)

- ▶ Also many of the patterns for safety are similar to what has already been covered
 - For instance, **isolation** applies also to safety; normal (or non-safety-critical) components (machines) should be separated from safety-critical components (machines)
- ▶ In addition, the **monitor-actuator** pattern can be mentioned as an example; it is generally a mixture of redundancy, sanity-checking, the barrier tactic, assertions, and alike

Safety Patterns (2/2)

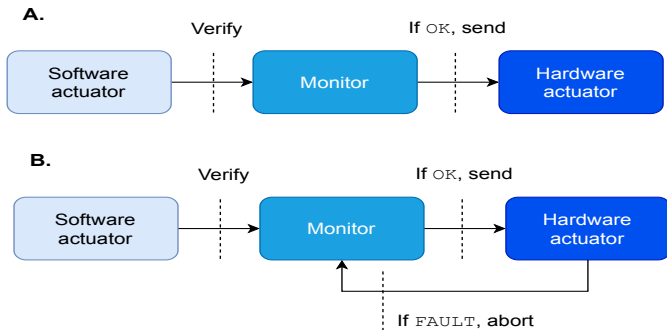


Figure 12: Two Sketches for the Monitor-Actuator Pattern

Security Scenarios (1/3)

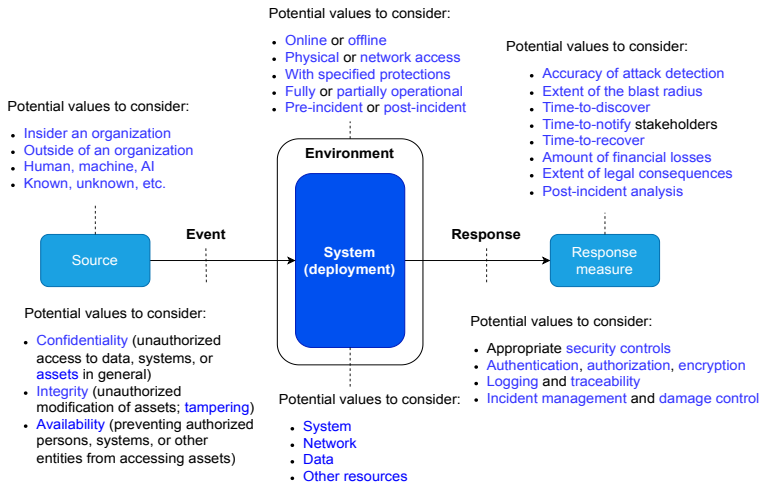


Figure 13: Adopted and modified from Bass et al. (2021)

Security Scenarios (2/3)

- ▶ Thus, the scenarios are quite different from those of other QAs
- ▶ Of the sources, it is worth emphasizing that the noun “actor” used earlier is still ultimately a human being, although AI and agents perhaps makes demarcations blurry in the future
 - Someone (or somebodies) still control a botnet, and so on
- ▶ Although not covered in this course, it also important to emphasize the distinction between internal and external sources; regarding the former, there are also **insider threats**
- ▶ As there is thus quite a lot to cover, we keep the focus on cyber security in relation to software architectures

Security Scenarios (3/3)

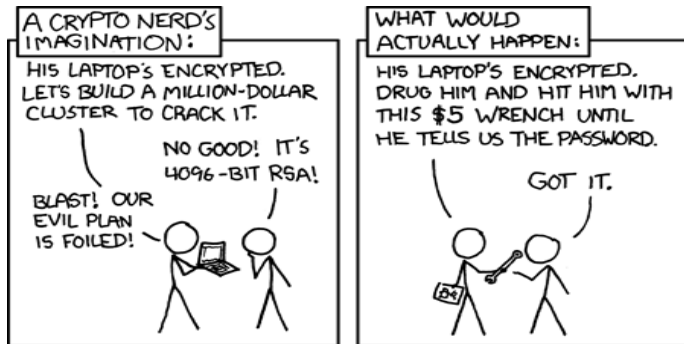


Figure 14: <https://xkcd.com/538/>

Security Tactics (1/2)

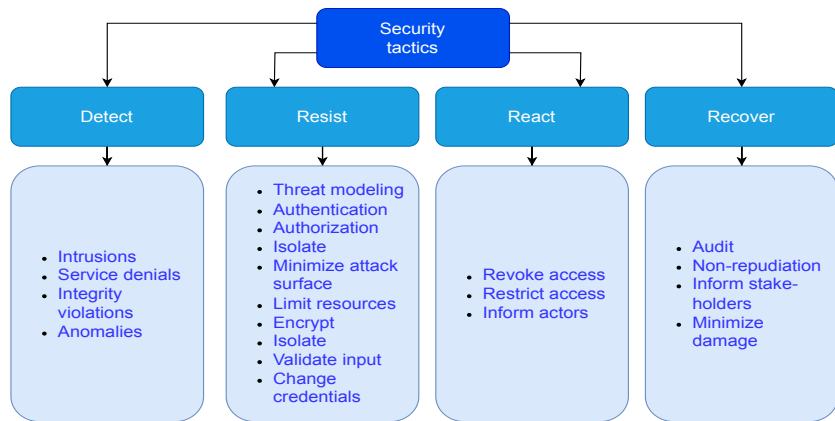


Figure 15: Adopted and modified from Bass et al. (2021)

Security Tactics (2/2)

- ▶ For this lecture, we will first exemplify the detect group with Security Information and Event Management (SIEM) systems
- ▶ Denial of service was already covered to a sufficient degree
- ▶ Intrusion detection systems (IDS) are not covered, although it is worth mentioning a lot of research in this area, including particularly with respect to deep learning models and AI
- ▶ Afterwards, the focus is on input validation and attack surface minimization as well as briefly on authentication and authorization, all addressed in the LLM/Agentic AI context

Security Tactics: Detect $\mapsto$ SIEM/SOC (1/4)

- ▶ SIEMs are essentially about managing security information of various kinds and integrating it to a management platform
 - The information to a SIEM may come from **internal sources** (such as IDS systems, spam filters, firewall logs, and many other systems, logs, and audit trails) as well as **external sources** (such as threat and vulnerability feeds)
- ▶ A term **security operations center** (SOC) is more or less used as a synonym to SIEM systems

Security Tactics: Detect $\mapsto$ SIEM/SOC (2/4)

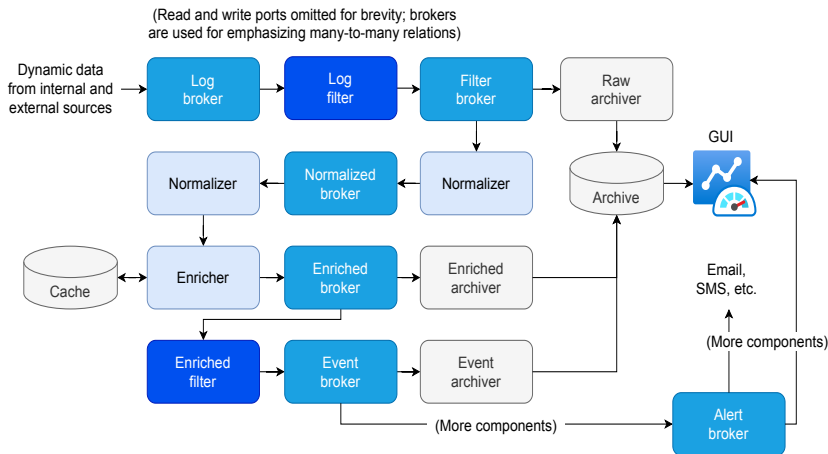


Figure 16: Adopted and simplified from Laue et al. (2022, Fig. 2)

A Question

What Pattern Is Used in the Previous Design?

Another Question

Where Might the Bottleneck in the Previous Design Be, and What Quality Attribute It Might Represent?

Security Tactics: Detect $\mapsto$ SIEM/SOC (3/4)



99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms

Bushra A. Alahmadi, Louise Axon, and Ivan Martinovic, *University of Oxford*

<https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi>

Figure 17: <https://www.usenix.org/system/files/sec22-alahmadi.pdf>

Security Tactics: Detect $\mapsto$ SIEM/SOC (4/4)

- ▶ To address the issues, **automation** is being increased
- ▶ As discussed by Vakulov (2026), operators of SIEMs/SOCs have two primary functions: (1) *to determine whether events are malicious*, and (2) *for true positives, execute appropriate action* (e.g., blocking a host or isolating a machine)
- ▶ Although automation via (1) still carries a lot of false positives, Vakulov (2026) argues that the workload of humans can be reduced also via (2) by increasing *operational autonomy* (e.g., execution of routine responses), *providing analytical assistance* (e.g., event prioritization), and *delegating decisions* (e.g., the system provides recommendations or even executes responses by itself)

LLM and Agentic Security (1/4)

- ▶ OWASP's (2024) somewhat old top-10 seems still reasonable:
 1. Prompt injection
 2. Information disclosure
 3. Supply-chains
 4. Data and model poisoning
 5. Improper output validation
 6. Excessive agency
 7. Prompt leakage
 8. Embedding weaknesses
 9. Misinformation
 10. Unbounded resource consumption
- ▶ Note that these may (even often) overlap, as soon seen

LLM and Agentic Security (2/4)

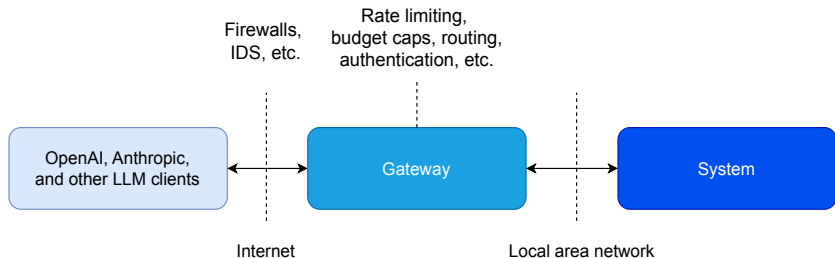


Figure 18: Where Is the Risk?

LLM and Agentic Security (3/4)



security

transparency

Incident Report: LiteLLM/Telnyx supply-chain attacks, with guidance

Figure 19: <https://blog.pypi.org/posts/2026-04-02-incident-report-litellm-telnyx-supply-chain-attack/>

LLM and Agentic Security (4/4)

- ▶ When it comes to unwanted information disclosure, data can leak also by traditional means, including in the AI industry



Figure 20: <https://www.theguardian.com/technology/2026/apr/01/anthropic-claudes-code-leaks-ai>

Input Validation (1/7)

- ▶ An overwhelming majority (perhaps around something like 99%) of attacks involve **malicious input** in a way or another
 - Recall that a TCP/IP packet is input to a machine too
- ▶ Therefore, always, always seek to **validate input** from untrusted sources; for that to be possible, you need to understand your system's **external-facing** components and the **propagation** of inputs through the system

Input Validation (2/7)

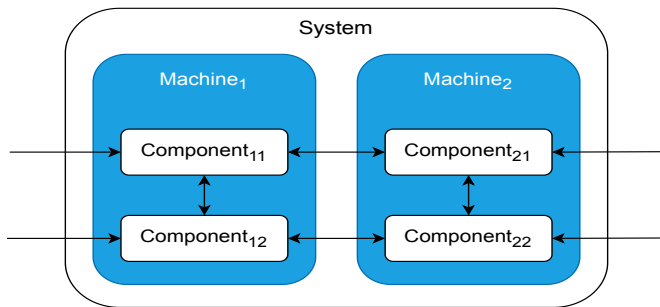


Figure 21: How to Improve the Security of the System?

Input Validation (3/7)

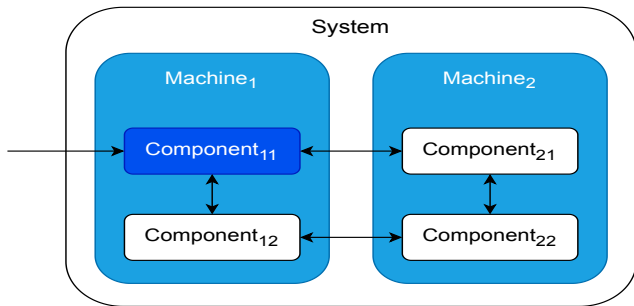


Figure 22: How to Still Improve the Security of the System?

Input Validation (4/7)

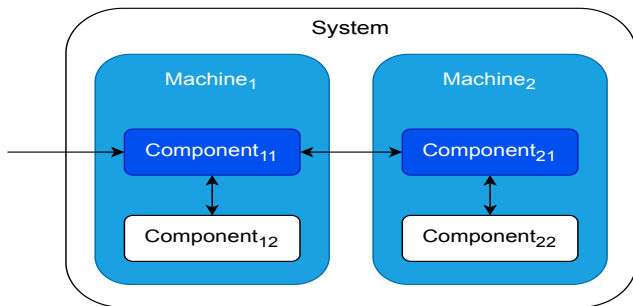


Figure 23: A Single Input Channel and Two Validating Components

Input Validation (5/7)

- ▶ The previous examples are not only about input as such but about the principle of **attack surface minimization**
- ▶ In general, the more there are inputs through external-facing components, the larger the attack surface
- ▶ Other QAs, such as **complexity**, affect also (or in particular from a security perspective) the external-facing components
 - If in the previous figure Component₁₁ has a million lines of code, say, the minimization of the input channels themselves may not necessarily matter

Input Validation (6/7)

03

Tool System

Every built-in tool Claude Code can call, sorted by what it does.

FILE OPERATIONS 6 tools	EXECUTION 3 tools	SEARCH & FETCH 3 tools	AGENTS & TASKS 10 tools	PLANNING 4 tools	MCP 4 tools	SYSTEM 4 tools	EXPERIMENTAL 4 tools
FileRead	Bash	WebFetch	Agent	EnterPlanMode	MCP	AskUserQuestion	Sleep
FileEdit	PowerShell	WebSearch	SendMessage	ExitPlanMode	ListMcpResources	TodoWrite	Brief
FileWrite	REPL	ToolSearch	TaskCreate	EnterWorktree	ReadMcpResource	Skill	SyntheticOutput
Glob			TaskGet	ExitWorktree	McpAuth	Config	LSP
Grep			TaskList			RemoteTrigger	
NotebookEdit			TaskUpdate			ScheduleCron	
			TaskStop				
			TaskOutput				
			TeamCreate				
			TeamDelete				

Bash

EXECUTION

Execute shell commands in the user's terminal with safety analysis

HOW IT WORKS

Runs commands through a safety analyzer that detects destructive operations (`rm -rf`, `git push --force`). Commands run in the user's shell environment with full access to installed tools.

PARAMETERS

`command` `timeout?`

`src/tools/BashTool/BashTool.ts`

Figure 24: How to **Not** Do It (from <https://ccunpacked.dev/>)

Input Validation (7/7)

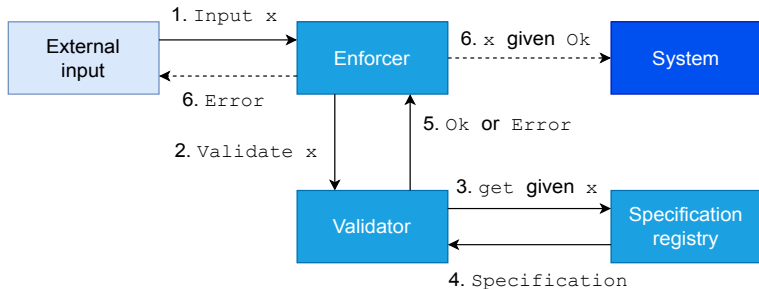


Figure 25: Input Validation With Three Components

Question

Where is the Risk in the Previous Figure?

Another Question

What Should the Enforcer Enforce With
the Error in the Previous Figure?

Design Principles for Input Validation (1/9)

- ▶ To put output validation aside for a moment, Arce et al. (2014) recommend to consider the following:
 1. Design and use a **centralized validation** component, but audit the component carefully and prefer well-known libraries
 2. Transform data into a **canonical form**; be aware of different encodings (cf. %-encoding in the Web, UTF-8, etc.)
 3. Recall that inputs often depend on **states**
 4. Audit all “**nearby code**” around inputs
 5. Prefer **strongly typed** and **memory-safe** languages

Input Validation (2/9)

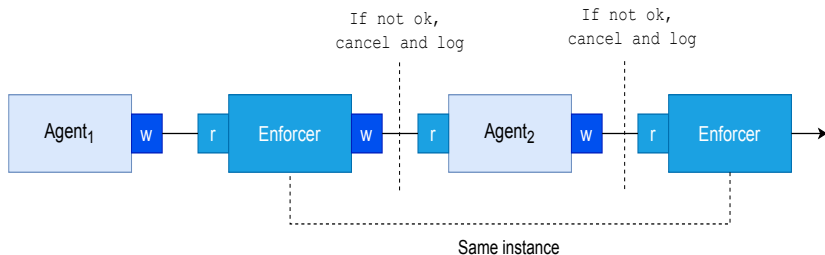


Figure 26: The Pipe and Filter Pattern (or the Batch-Sequential Pattern) With Input (and Output) Validation

Design Principles for Input Validation (3/9)

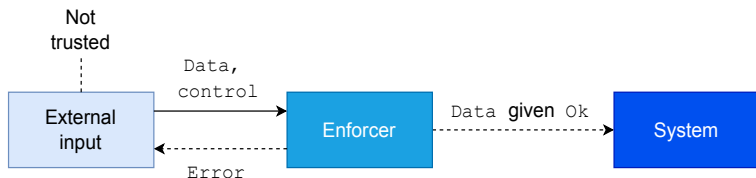
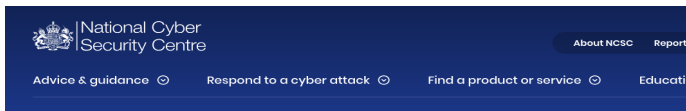


Figure 27: In Addition, Arce et al. (2014) Recommend to *Always Separate Data from Control Instructions and Never Process Control Instructions from Untrusted Sources*

Design Principles for Input Validation (4/9)



BLOG POST

Prompt injection is not SQL injection (it may be worse)

There are crucial differences between prompt and SQL injection which – if not considered – can undermine mitigations.

Summary

Whilst the comparison of prompt injection to SQL injection can be tempting, it's also dangerous. SQL injection **can** be properly mitigated with parameterised queries, but there's a good chance prompt injection will **never** be properly mitigated in the same way. The best we can hope for is reducing the likelihood or impact of attacks.

Figure 28:

<https://www.ncsc.gov.uk/blog-post/prompt-injection-is-not-sql-injection>

Design Principles for Input Validation (5/9)

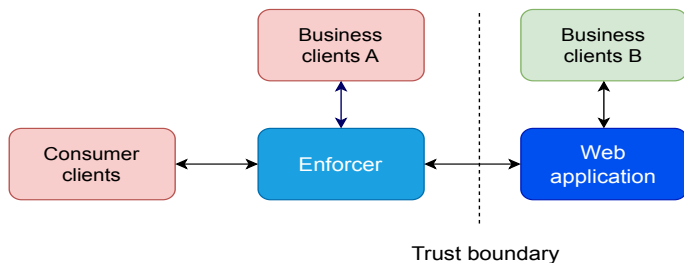


Figure 29: They Also Recommend to *Audit the Trust Boundaries*

Design Principles for Input Validation (6/9)

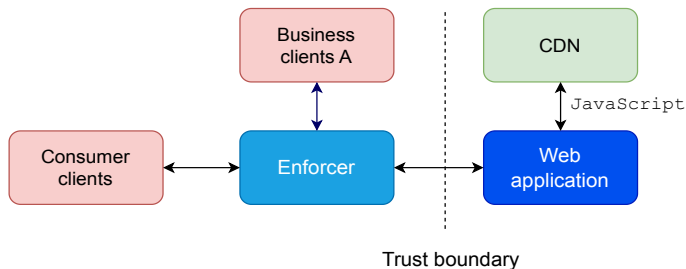


Figure 30: Where Is the Risk?

Design Principles for Input Validation (7/9)

Subresource Integrity

W3C Working Draft, 20 March 2026



▼ More details about this document

This version:

<https://www.w3.org/TR/2026/WD-sri-2-20260320/>

Latest published version:

<https://www.w3.org/TR/sri-2/>

Editor's Draft:

<https://w3c.github.io/webappsec-subresource-integrity/>

Previous Versions:

<https://www.w3.org/TR/2026/WD-sri-2-20260318/>

History:

<https://www.w3.org/standards/history/sri-2/>

Feedback:

public-webappsec@w3.org with subject line "[sri] ... message topic ..."
([archives](#))
[GitHub](#)

Editor:

[Frederik Braun](#) (Mozilla) fbraun@mozilla.com

Former Editors:

[Devdatta Akhawe](#) (Dropbox Inc.) dev.akhawe@gmail.com

[François Marier](#) (Mozilla) francois@mozilla.com

[Joel Weinberger](#) (Google Inc.) jww@google.com

Test Suite:

<https://wpt.fyi/results/subresource-integrity/>

Copyright © 2026 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Figure 31: <https://www.w3.org/TR/sri-2/>

Design Principles for Input Validation (8/9)

Content Security Policy Level 3

W3C Working Draft, 11 March 2026



▼ More details about this document

This version:

<https://www.w3.org/TR/2026/WD-CSP3-20260311/>

Latest published version:

<https://www.w3.org/TR/CSP3/>

Editor's Draft:

<https://w3c.github.io/webappsec-csp/>

History:

<https://www.w3.org/standards/history/CSP3/>

Feedback:

public-webappsec@w3.org with subject line "[CSP3] ... message topic ..." ([archives](#))
[Github](#)

Editors:

[Mike West](#) (Google Inc.)
[Antonio Sartori](#) (Google Inc.)

Participate:

[File an issue](#) ([open issues](#))

Tests:

[web-platform-tests content-security-policy/](#) (ongoing work)

Copyright © 2026 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Figure 32: <https://w3c.github.io/webappsec-csp/>

Design Principles for Input Validation (9/9)

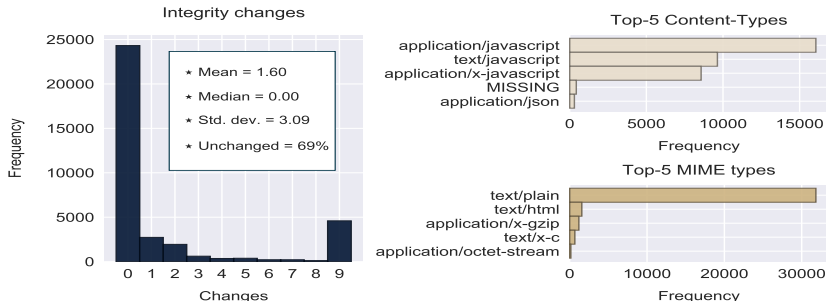


Figure 33: Changes of Dynamically Loaded External JavaScript Code in a Ten Day Polling Period of Circa 35 Thousand Popular Websites (from Ruohonen et al. 2018, Fig. 1)

An Example of GitHub Actions (1/6)

- ▶ GitHub Actions are a good example about many of the previous points on input validation; they are development **workflows** that can run in response to various **events** (such as pull requests); they can be triggered by many **actors**; they can carry **secrets**; and they can run with different **permissions**
- ▶ Given the previous characterization, the attack surface is indeed large, and in response to recent incidents, Microsoft is currently, in 2026, seeking to improve their security

An Example of GitHub Actions (2/6)

- ▶ As described by Ose & Glass (2026), an obvious starting point for reducing the attack surface is to define **who can trigger** workflows and **which events** are allowed to be triggered
 - These questions are about the **restrict access tactic**
- ▶ Does CoPilot or a LLM from GitHub Apps really need permissions for triggering all (or even any...) workflows?
- ▶ In terms of security engineering principles, defining these is about **secure defaults**, which, in most cases, should follow a “**deny all**” principle (“deny everything, then allow particular”)

An Example of GitHub Actions (3/6)

- ▶ As further described by Ose & Glass (2026), the attack surface of GitHub Actions can also be reduced by increasing the **granularity of permissions** and access controls
- ▶ For this to be possible, the **trust boundaries must be explicit**; instead of having coarse repository-wide or organization-wide secrets, these are tied to specific workflows
- ▶ It must be also ensured that **secrets are not inherited** unless explicitly allowed (e.g., a secret can be scoped only to a specific path or a given workflow can has its own secret)
 - This connotes with the **least privilege** principle

An Example of GitHub Actions (4/6)

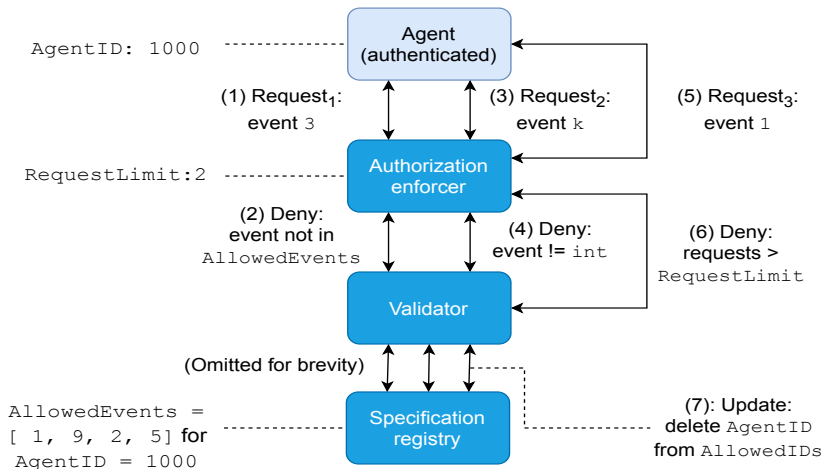


Figure 34: A Simplified Sketch for the Revoke Access Tactic

An Example of GitHub Actions (5/6)

- ▶ Finally, Ose & Glass (2026) also note that they are also moving toward **egress filtering** for the CI/CD pipelines
- ▶ Theoretically, egress filtering is about **output validation**; in terms of GitHub, it means defining to which domain names or IP addresses connections can be made, which HTTP methods are allowed and with which TLS configurations, and so forth
- ▶ Egress filtering can be recommended also for meeting the **CRA**'s requirements because the regulation specifies that a device should not harm its operating networking environment

An Example of GitHub Actions (6/6)



OpenAI patches ChatGPT flaw that smuggled data over DNS

Check Point says outbound controls blocked web traffic but overlooked DNS

 [Thomas Claburn](#)

Mon 30 Mar 2026 // 19:36 UTC

Figure 35: https://www.theregister.com/2026/03/30/openai_chatgpt_dns_data_snuggling_flaw/

Question

How Many Trust Boundaries for
Agentic AI Can You Quickly Name?

Some (But Hardly All!) Trust Boundaries for Agentic AI

1. User $\leftrightarrow$ Agent
2. Agent $\leftrightarrow$ Tool
3. Tool $\leftrightarrow$ Supply-chain
4. Execution $\leftrightarrow$ Tool
5. Agent $\leftrightarrow$ Content
6. Execution $\leftrightarrow$ Content
7. Runtime $\leftrightarrow$ Execution
8. Host $\leftrightarrow$ Runtime
9. Memory $\leftrightarrow$ Runtime
10. Host & memory $\leftrightarrow$ Kernel
11. $\vdots$

Another Question

If You Look at the Previous List Again,
What Trust Boundaries Would You Say Are
Relatively New in Terms of Cyber Security?

With Agents (and RAGs), What Really is Executable?



AI supply chain attacks don't even require malware...just post poisoned documentation

A proof-of-concept attack on Context Hub suggests there's not much content sanitization

 [Thomas Claburn](#)

Wed 25 Mar 2026 // 20:50 UTC

A new service that helps coding agents stay up to date on their API calls could be dialing in a massive supply chain vulnerability.

Figure 36: https://www.theregister.com/2026/03/25/ai_agents_supply_chain_attack_context_hub/

Rethinking About the OWASP's Top-10 (1/2)

- ▶ Thus, we should stop for a moment to really rethink the earlier top-10 listing because it is possible to think about sequences leading to execution, such as “*Agent* $\rightarrow$ *Get [malicious] content* $\mapsto$ *Get a [malicious] tool based on the [malicious] content* $\rightarrow$ *Execute the [malicious] tool*”
- ▶ But how to categorize data poisoning then? Is it the malicious content that is poisoned? Or should we consider also malware disguised as legitimate tools as data poisoning of sorts?

Rethinking About the OWASP's Top-10 (2/2)

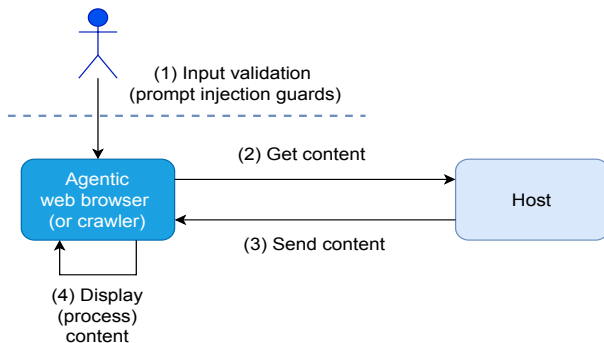


Figure 37: What Could Go Wrong and How to Try to Make it Better?

Rethinking About the OWASP's Top-10 (3/3)

- ▶ Yet, with recent introduction of so-called agentic **skills**, the trust boundaries extend further; e.g., (1) Agent $\leftrightarrow$ Task, (2) Task $\leftrightarrow$ Skill, (3) Execution $\leftrightarrow$ Skill
- ▶ As described by Didi & Zavodchik (2026) from Akamai, these skills for AI agents bring a whole group new attack vectors
 - Among these is also skill-to-skill propagation (and, hence, execution propagation), and, therefore, the earlier point about extending the batch-sequential pattern is a good idea
 - They also demonstrate the weaknesses of **service discovery** pattern, including with respect to software supply-chains

Rethinking Agents as Distributed Systems

- ▶ All of the points from the previous three lectures apply
- ▶ For instance, how other agents can detect a **crash** of an agent and continue their work without the crashed agent?
- ▶ How about **synchronization**: if one agent writes to a file (e.g., fixes a bug) and other agent reads from the file?
- ▶ How about the **CAP theorem**; when a partitioning occurs, how can agents synchronize their states without losing their work in the process?
- ▶ As far as I understand, all these are still unsolved issues (from a scientific perspective) at the time of lecturing

Input and Identities (1/5)

- ▶ The GitHub Actions case is illuminating in a sense that it shows that **identities** for authentication must be extended beyond human users; workflows, tools, hosting services, and other elements should also come with identities
- ▶ Yet, Sierra (2026) argues for the following two sensible design principles for authorization in agentic AI systems:
 1. *“Authorization must not depend on the model’s interpretation of the request.”* ($\implies$ potential prompt injection)
 2. *“Authorization must be enforced by deterministic system controls at trust boundaries”* ($\implies$ an agent cannot authorize another agent and trust boundaries must be made explicit)

Input and Identities (2/5)

- ▶ Note that in cyber security identities must always satisfy **non-repudiation**; actions cannot be disputed by an actor and origins of data come with proof about the integrity of the data
 - Hence, why logging is done for security purposes
- ▶ In terms of agentic AI, non-repudiation generally means that it should be possible to bound all actions to a **real user** with clear provenance

Input and Identities (3/5)

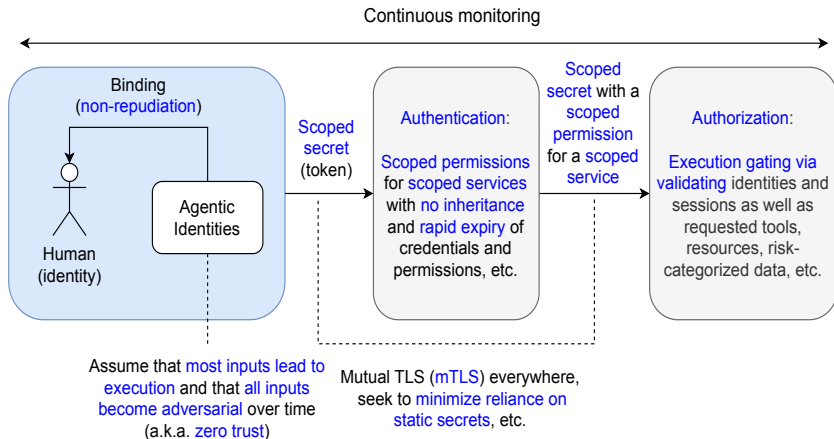


Figure 38: Some Tentative Principles for Securing AI Agents (Sierra 2026)

Input and Identities (4/5)

draft-hartman-credential-broker-4-agents-00

Network Working Group
Internet-Draft
Intended status: Informational
Expires: 30 September 2026

K. Hartman
SANS Institute
29 March 2026

Credential Broker for Agents (CB4A)
draft-hartman-credential-broker-4-agents-00

Abstract

This document specifies a Credential Broker for Agents (CB4A). a credential vaulting and brokering architecture that mediates AI agent access to API credentials. Agents never hold real long-lived credentials. Instead, they receive short-lived, narrowly scoped, auditable proxy credentials issued by a broker that separates policy decisions from credential delivery. The architecture addresses the "credential sprawl" risk inherent in agentic AI, where agents aggregating access across many services become high-value compromise targets.

Figure 39: <https://datatracker.ietf.org/doc/draft-hartman-credential-broker-4-agents/>

Input and Identities (5/5)

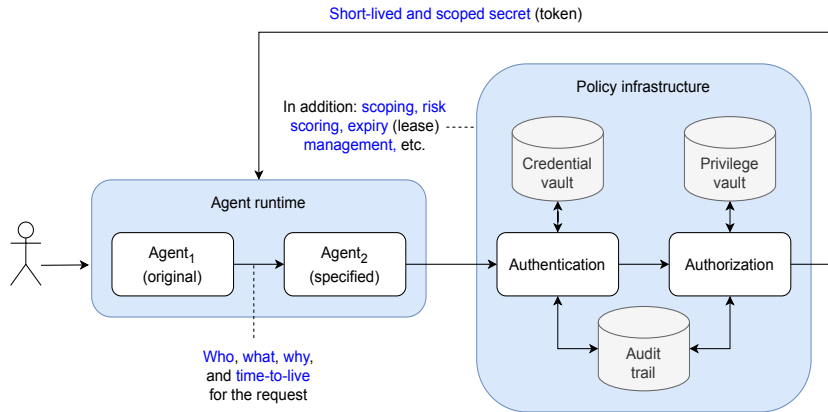


Figure 40: Further Emerging Ideas for Agent Security (loosely adopted from the previous CB4A draft standard)

Input and Privileges (1/8)

- ▶ As already noted, the **principle of least privileges** means that a software should only operate with the minimal privileges it requires for its functioning
 - Theoretically, it too can be seen as an attack surface minimization technique; a restricted process with minimal privileges can do much less harm than an unrestricted one
- ▶ The principle is particularly important when processing input from untrusted sources!
- ▶ Note that these principles align with the performance tactic of **bound execution** (i.e., limits, caps, thresholds, etc.)

Input and Privileges (2/8)

- ▶ **Dropping privileges** after a process has started is a common and old security design technique (Chen et al. 2002)
 - It was allegedly first used by Bernstein with *qmail* in 1997; thereafter, many Unix/Linux programs and services have been refactored to drop privileges from the root user
- ▶ However, not all programs and services can entirely drop the root privilege; therefore, another technique is to **separate privileges** by isolating all risky areas to non-root process(es)
 - A beautiful example of the technique is *Postfix*

Input and Privileges (3/8)

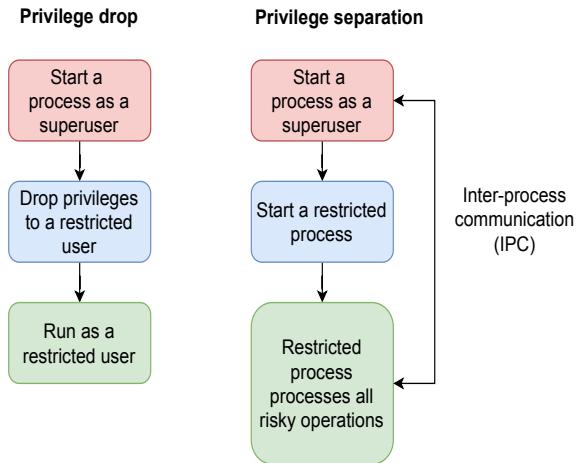


Figure 41: Privilege Drop and Separation in a Nutshell

Input and Privileges (4/8)

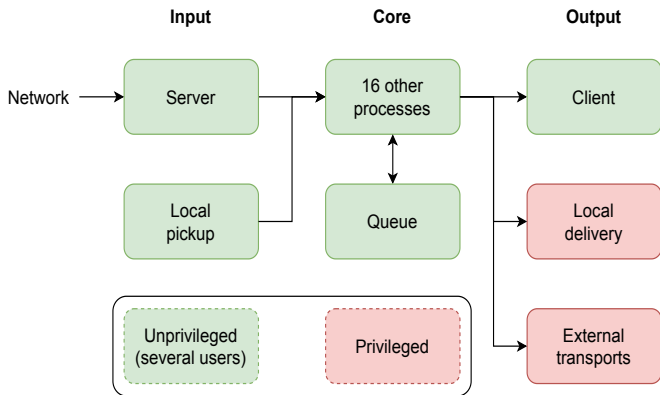


Figure 42: Privilege Drop and Separation in Postfix (Venema 2010)

Input and Privileges (5/8)

- ▶ The same classical – and battle-tested – principles can be applied also to containers and Kubernetes pods covered in the previous lecture; for instance, Kubernetes (2026) can be configured akin to:

```
kind: Pod
metadata:
  name: example-pod
spec:
  securityContext:
    runAsUser: 1000
    runAsGroup: 2000
  [...]
```

Input and Privileges (6/8)

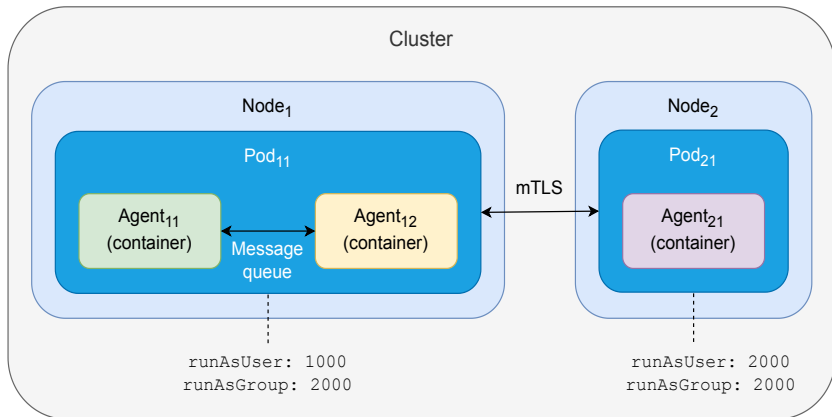


Figure 43: Privilege Drop with Kubernetes

Input and Privileges (7/8)

PLEDGE(2)

System Calls Manual

PLEDGE(2)

NAME

pledge — restrict system operations

SYNOPSIS

```
#include <unistd.h>
```

```
int
```

```
pledge(const char *promises, const char *execpromises);
```

DESCRIPTION

The **pledge()** system call separates the POSIX featureset into a group of approximately 3 dozen subsystems. By calling **pledge()** the program can declare which subsystems it will need in the future in a space-separated string called *promises*. Subsystems not listed become unavailable, and most attempts to use operations in that subsystem result in the process being killed with an uncatchable SIGABRT, delivering a core file if possible.

Figure 44: <https://man.openbsd.org/pledge>

Input and Privileges (8/8)

- ▶ Once you have implemented the least privilege principle, Adkins et al. (2020) from Google recommend testing your implementation with respect to “**of privileges**” (i.e., verifying the implementation) and “**with privileges**” (i.e., verifying the implementation’s execution)
- ▶ In addition, they recommend considering a **breakglass mechanism**; sometimes someone may need to intentionally break your security mechanisms in the face of emergencies (e.g., a physician may need to break access controls in emergency situations in order to save a patients’ lives)

Questions?

Case #8

- ▶ Sketch a faulty distributed system with which three AI agents collaborate over a network. One agent should analyze code and write bug reports; another agent should do bug fixes and make pull requests; and the third agent should approve and merge the requests. The emphasis is on the adjective *faulty*; the sketch should demonstrate at least three *architectural* security weaknesses and two other *architectural* weaknesses related to other quality attributes addressed thus far in the course. You can elaborate the weaknesses in writing (and orally in the class), with diagrams, or both.

Additional References (1/2)

1. Laue, T. & Klecker, T. & Kleiner, C. & Detken, K-O. (2022): A SIEM Architecture for Advanced Anomaly Detection. Open Journal of Big Data 6 (1): 26–42, https://serwiss.bib.hs-hannover.de/frontdoor/deliver/index/docId/2321/file/laue_etal2022-siem_arch_anomaly_detection.pdf
2. Arce, I. et al. (2014): Avoiding the Top 10 Software Security Design Flaws. IEEE, <https://ieeecs-media.computer.org/media/technical-activities/CYBSI/docs/Top-10-Flaws.pdf>
3. Ruohonem, J. & Salovaara, J. & Leppänen, V. (2018): On the Integrity of Cross-Origin JavaScripts. IFIP SEC, <https://arxiv.org/pdf/1809.05628>
4. Ose, G. & Glass, S. (2026): What's Coming to our GitHub Actions 2026 Security Roadmap. GitHub, Inc. <https://github.blog/news-insights/product-news/whats-coming-to-our-github-actions-2026-security-roadmap/>
5. OWASP (2024): OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>

Additional References (2/2)

1. Vakulov, A. (2026): Delegating Decisions in Security Operations. Communications of the ACM, <https://cacm.acm.org/blogcacm/delegating-decisions-in-security-operations/>
2. Sierra, P. D. (2026): AI Security: Identity and Access Control. Red Hat Blog, <https://www.redhat.com/en/blog/ai-security-identity-and-access-control>,
3. Didi, T. & Zavodchik, M. (2026): The Top 10 Threats Related to Agent Skills. Akamai Security Blog, <https://www.akamai.com/blog/security/2026/mar/top-10-threats-related-agent-skills>
4. Chen, H. & Wagner, D. & Dean, D. (2002): Setuid Demystified. USENIX Security Symposium, pp. 1–21, https://www.usenix.org/legacy/publications/library/proceedings/sec02/full_papers/chen/chen.pdf
5. Venema, W. (2010): Postfix, Past Present and Future. Invited talk at USENIX Security Symposium, <https://www.usenix.org/legacy/event/lisa10/tech/slides/venema.pdf>
6. Kubernetes (2026): Configure a Security Context for a Pod or Container. <https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>